

Chapter 3 · Lesson 2 — Rendering the Standings

Chapter 3 · Lesson 2 — Rendering the Standings

Goal: build the first real, data-driven React component in this app — `<StandingsTab>` v1. By the end of this lesson, you'll see a sortable table of 8 fantasy teams in your browser, ranked by total points.

The smallest version that proves data flows

When you start a new feature in React, the first version should be **the smallest thing that proves the data is real**. For the standings, that's just team names and totals — no styling, no icons, no medals, no per-round columns. Just rows and a number.

We'll build that v1 first, see it work, then iterate. **This is the way.**

Why? Because if your first version has 200 lines of styling AND data binding AND sorting AND conditional rendering, and something doesn't work, you have to debug *all of it at once*. A 15-line v1 lets you find data-binding bugs in 5 seconds. Then you add styling on top, debug only the styling, and move on. **One concern at a time.**

Step 1: stub the orchestrator

Before the standings tab can render, the page needs *something* to render. Create `components/SnookerFantasyLeague.tsx` with a stub:

```
'use client';

import StandingsTab from '../tabs/StandingsTab';

export default function SnookerFantasyLeague() {
```

```

return (
  <div style={{ padding: 32, fontFamily: 'sans-serif' }}>
    <h1>Snooker Fantasy League</h1>
    <StandingsTab />
  </div>
);
}

```

Five things to notice:

'use client' at the top – finally

This is the directive we've been foreshadowing. Putting 'use client' as the first line of a file marks the file (and everything it imports that's not also marked server) as a **client component**. That means it gets shipped to the browser as JavaScript and can use `useState`, `useEffect`, event handlers, etc.

Why does `<SnookerFantasyLeague>` need to be a client component? Because in the next chapter we'll add `useState` to track which tab is active. State requires client-side hydration. Pre-marking it now saves a refactor later.

Does this make EVERYTHING client-side now? Yes – but only the `SnookerFantasyLeague` subtree, which is most of the app. The `app/layout.tsx` and `app/page.tsx` are still server components and still render to HTML on the server. That HTML wraps the client JS bundle. So the page is still **statically prerenderable** (no real-time data) and benefits from server rendering for SEO, while still allowing client-side interactivity.

This is the App Router's superpower: **mix server and client at any boundary**. The boundary here is `<SnookerFantasyLeague>`. Above it (in `layout.tsx` and `page.tsx`) – server. Inside it – client.

Importing from a sub-path

```
import StandingsTab from './tabs/StandingsTab';
```

The relative import `./tabs/StandingsTab` works because we're inside `components/` and `StandingsTab.tsx` is at `components/tabs/StandingsTab.tsx`.

We *could* also use the path alias: `import StandingsTab from '@/components/tabs/StandingsTab'`. Both work. The codebase uses relative imports for *intra-folder* references and `@/` aliases for *cross-folder* references (e.g. `@/lib/teams`). It's a soft convention that keeps imports local-feeling.

Stub style — minimal so the data is the focus

```
<div style={{ padding: 32, fontFamily: 'sans-serif' }}>
```

This wrapper exists purely so the page isn't visually flush against the browser edge. We'll replace it with the gradient hero in Chapter 4.

Step 2: the v1 StandingsTab

Create `components/tabs/StandingsTab.tsx`:

```
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';

export default function StandingsTab() {
  const teamsWithScores = TEAMS.map(t => ({
    ...t,
    scores: calculateTeamScores(t),
  })).sort((a, b) => b.scores.total - a.scores.total);

  return (
    <table>
      <thead>
        <tr>
          <th>Rank</th>
          <th>Team</th>
          <th>Total</th>
        </tr>
      </thead>
      <tbody>
        {teamsWithScores.map((team, i) => (
          <tr key={team.name}>
```

```

        <td>{i + 1}</td>
        <td>{team.name}</td>
        <td>{team.scores.total}</td>
      </tr>
    )))
  </tbody>
</table>
);
}

```

Run `npm run dev` and visit `http://localhost:3000`. You should see a table with 8 rows showing team names ranked by total points.

It looks ugly. **That's fine.** It works.

What's happening in this file

Walk through each piece.

Line 1–2: imports

```

import { TEAMS } from '@lib/teams';
import { calculateTeamScores } from '@lib/scoring';

```

We're pulling the data and the scoring function from the `lib/` folder we built in Chapter 2. The `@/` is the path alias from `tsconfig.json`. **Notice there's no `import React from 'react'` at the top** — modern JSX (since React 17) doesn't require it. The compiler imports React automatically when needed. You'll only `import { useState, useMemo } from 'react'` for hooks.

Line 4: the function

```

export default function StandingsTab() {

```

No props. The component uses imported data directly, with no parent involvement. That's fine for a leaf component that owns its data.

Line 5–8: derive the data

```
const teamsWithScores = TEAMS.map(t => ({
  ...t,
  scores: calculateTeamScores(t),
})).sort((a, b) => b.scores.total - a.scores.total);
```

This is the most important line in the file. Read it twice.

Step 1: `TEAMS.map(t => ({ ...t, scores: calculateTeamScores(t) })))` For each team, produce a new object that: - Spreads the team’s existing fields (name, color, r1, r2, ...): `...t` - Adds a new scores field computed from `calculateTeamScores(t)`.

The result is an array of “team-with-scores” objects — the original team plus a scores object containing all the round totals and detail arrays.

The `...t` **spread** is JavaScript shorthand for “copy all of t’s fields into the new object.” Without it we’d have to type out every field manually:

```
{ name: t.name, color: t.color, accent: t.accent, /* etc */ scores: calculateTeamScores(t) }
```

Spread is shorter and more maintainable. If a new field is added to Team later, the spread automatically picks it up.

Step 2: `.sort((a, b) => b.scores.total - a.scores.total)` Sort by `scores.total` descending. Because `b - a` is positive when `b` is bigger than `a`, `.sort()` puts bigger values first. **This is the standard “sort descending by a number” recipe.**

If you wanted ascending instead, you’d write `a - b`.

Why is sorting OK here? `.sort()` mutates the original array. That’s a footgun in React if you’re not careful — sorting a prop array would mutate the parent’s data. But here, **`TEAMS.map(...)` produces a brand new array** that we then sort. The original `TEAMS` is untouched. **Mutating an array you just made is fine.** Mutating an array someone else gave you is not.

Why is this in the function body, not at the module level? We could move the `teamWithScores` calculation outside the component:

```
const teamsWithScores = TEAMS.map(...).sort(...);    // module-level

export default function StandingsTab() {
  return ( /* ... */ );
}
```

It would still work. It would even be slightly faster (computed once, not per-render). But:

- For Chapter 4, we'll need this calculation inside the component so we can pass `teamsWithScores` to other tabs from the orchestrator.
- For now, having it inside is fine — `calculateTeamScores` for 8 teams is microseconds.
- In Chapter 4 we'll wrap it in `useMemo` to make the per-render cost negligible.

We're optimizing for **readability and refactor-ability**, not raw speed.

Lines 11–22: the JSX

```
return (
  <table>
    <thead>
      <tr><th>Rank</th><th>Team</th><th>Total</th></tr>
    </thead>
    <tbody>
      {teamsWithScores.map((team, i) => (
        <tr key={team.name}>
          <td>{i + 1}</td>
          <td>{team.name}</td>
          <td>{team.scores.total}</td>
        </tr>
      ))}
    </tbody>
  </table>
);
```

Standard HTML `<table>` markup. Three things to dig into.

The key prop and why it matters

Look at this line:

```
{teamsWithScores.map((team, i) => (  
  <tr key={team.name}>
```

That `key={team.name}` is **not optional**. It's React's identity tracker for list items.

What is React doing?

When the data changes — say, a team's score goes up and the order shifts — React needs to figure out which DOM nodes to keep, which to update, and which to throw away. Without keys, React falls back to comparing by **position in the array**: the first row is the first row, the second is the second, etc.

That's a problem if the list re-orders. If "Invincibles" was rank 1 and "BBU" was rank 2, then BBU jumps to rank 1 — without keys, React thinks "the first row is BBU now (was Invincibles); update its content. The second row is Invincibles now (was BBU); update its content." It works, but **every row's content is replaced**, even though the rows just swapped.

With keys, React thinks "the row with key='BBU' moved from position 2 to 1. The row with key='Invincibles' moved from position 1 to 2. Just rearrange them." Much faster, no re-rendering of contents, animations work correctly.

What makes a good key?

Three rules:

1. **Stable** — the same item gets the same key across renders. `team.name` is stable. `Math.random()` is the worst possible key.
2. **Unique within the list** — no two siblings share a key. Two teams can't have the same name (in our app), so `team.name` is fine. If you had duplicate-able names, use an `id` field.
3. **Predictable** — derived from the data, not from external state. `i` (the index) works **only** if the list never reorders or has items inserted/deleted. For a sortable list, `i` is wrong. **For our standings, `team.name` is right.**

The classic key bug

Beginners often write `key={i}` because the index is right there. It works **right up until the list reorders or has an item inserted in the middle**, at which point React applies updates to the wrong rows and you get bizarre bugs (form inputs lose their values, animations attach to the wrong row, etc.).

Rule of thumb: **use `i` only when the list is append-only and never sorted**. Otherwise use a stable id.

Interview question: “What does the *key prop* do?” The wrong answer is “it makes the warning go away.” The right answer is “it tells React the identity of each list item across renders, so React can reconcile the DOM correctly when the list changes.”

Embedding expressions inside JSX

```
<td>{i + 1}</td>
<td>{team.name}</td>
<td>{team.scores.total}</td>
```

The `{...}` lets you embed any JavaScript expression. Three different things appear:

- `i + 1` — arithmetic. We add 1 because `.map`’s `i` is 0-indexed but humans expect ranks starting at 1.
- `team.name` — property access on an object.
- `team.scores.total` — nested property access.

All three are JS expressions. Anything that evaluates to a value works inside `{}`.

What **doesn’t** work: statements. You can’t write `{if (foo) ...}` because `if` is a statement, not an expression. You’d use a ternary instead: `{foo ? a : b}`.

Returning JSX vs returning null

What if we want to render nothing in some condition? React lets you return:

- JSX
- A string or number (rendered as text)
- `null` (renders nothing)

- `false` (also renders nothing)
- An array of JSX (renders all of them – but every item needs a key)

```
return null; // renders nothing
return <p>Hello</p>; // renders a paragraph
return condition ? <p>Yes</p> : null; // conditional
```

Returning `null` is normal. It's not an error – it just means “don't render anything for this component this time.”

This comes up later when, e.g., a `<Modal>` returns `null` if it's not open.

Step 3: visit your browser

After saving both files:

1. Make sure `npm run dev` is running.
2. Open `http://localhost:3000`.
3. You should see your stub heading and the table below it.

If you see something different:

- **Blank page or error?** Check the terminal – Next.js will print compilation errors. The most common is “Cannot find module ‘@/lib/teams’” (typo in path) or “TEAMS is not exported” (forgot `export` in `lib/teams.ts`).
- **Table renders but all totals are 0?** Likely the matches don't have winner filled in, or the picks contain typos. Add a `console.log(teamsWithScores)` temporarily and inspect.
- **Hydration failed because the initial UI does not match?** Server and client rendered different things. Common cause: using `Math.random()` or `Date.now()` directly in render. Don't.

Why this is “the right pace” for a beginner

You've now written **20 lines of React** that:

- Imports data from a typed module.
- Transforms it via `.map()` and `.sort()`.
- Renders it as a table with proper keys.
- Lives inside a Next.js client-component subtree.

If a friend asked you “what’s React?” tonight, you could legitimately say “it’s a way to derive a new data structure from the original data and tell the browser to render it as HTML, with React managing the diffing.” That’s a senior-level mental model, in 20 lines.

The rest of the chapter (and the rest of the course) is just **more of the same pattern, with more sophisticated data transforms and JSX**. There’s no new fundamental concept after this; everything else is variations on a theme.

Vibe prompt you would have used

“Create components/tabs/StandingsTab.tsx. Import TEAMS from @/Lib/teams and calculateTeamScores from @/Lib/scoring. Build a teamsWithScores array by mapping over TEAMS and adding scores: calculateTeamScores(t) to each, then sort descending by .scores.total. Render a plain <table> with thead Rank/Team/Total and a tbody mapping over teamsWithScores. Each row needs key={team.name}. No styling, no inline styles, just structural HTML. Default export.”

The “no styling” line is the secret. Without it the LLM will give you 80 lines of beautiful CSS and you won’t be able to tell what’s broken when something is broken.

CHECK YOURSELF

1. `<tr key={team.name}>` – what would happen if you used `<tr key={i}>` instead? Walk through what React does when you change the sort order. (Hint: think about `<input>` fields if any of the rows had editable cells.)
2. Why is `'use client'` on `SnookerFantasyLeague.tsx` but not on `StandingsTab.tsx`? What determines which files need it?
3. `TEAMS.map(t => ({ ...t, scores: ... })).sort(...)` – does this mutate the original `TEAMS` array? Why or why not?
4. You add a `console.log` inside the component body, just before return. When does it run? What if the user clicks a button somewhere on the page that doesn’t affect this component – does the log run again?
5. Add a `<th>R1</th>` to the header and a `<td>{team.scores.r1}</td>` to each row. What’s the minimum change? Now the page shows R1 totals next to the grand total. (Don’t move on until you’ve actually done this.)

When you’ve answered #5 by typing the change and seeing it work, you’re ready for Lesson

3 — adding the rest of the columns and the visual polish.

Head to [03-progressive-enhancement.md](#).